

</> StudentLive.tsx.tsx
TypeScript

</> StreamLive.tsx
TypeScript

</> StreamHeader.tsx
TypeScript

</> useLiveRoom.ts
TypeScript

porque n está funcionando? o aluno n recebe os dados de video e audio do professor... o resto funciona

Sim. Olhando os 4 arquivos, o problema principal está na **senalização WebRTC entre professor e aluno**, não no Supabase nem no `<video>` .

Há dois problemas importantes:

1. O professor cria um `offer` para o aluno quando aprova, mas **não existe nenhum listener para receber o `answer` do aluno**.
2. A conexão é criada/negociada em dois lugares diferentes (`useLiveRoom` e `StreamLive`), o que deixa a `RTCPeerConnection` em estados de negociação conflitantes.

O erro mais direto

No professor, quando você aprova o aluno:

```
const offer = await pc.createOffer();  
await pc.setLocalDescription(offer);
```

```
channelRef.current.send({  
  type: 'broadcast',  
  event: 'offer',
```

</> TypeScript



Isso está em `StreamLive.tsx` .  `StreamLive`

O aluno recebe esse `offer` , faz:

```

await pc.setRemoteDescription(new RTCSessionDescription(payload.offer));

const answer = await pc.createAnswer();
await pc.setLocalDescription(answer);

roomChannel.send({
  type: 'broadcast',

  </> TypeScript
});

```



Ou seja, o aluno **manda o answer corretamente**. StudentLive.tsx

Só que no professor **não há nenhum**:

```

</> TypeScript

```



O `useLiveRoom.ts` trata `student-media-offer`, `ice-candidate`, `join-request`, etc., mas não trata `answer`. `useLiveRoom`

Então fica assim:

```

PROFESSOR                                ALUNO

createOffer()
|
| ----- offer -----> |
|                         |
|                         | setRemoteDescription()
|                         |
|                         | createAnswer()
|                         |
| <----- answer -----|
|
X ninguém processa o answer

```



Consequentemente, a negociação do professor → aluno não termina corretamente.

Tem um segundo problema ainda mais importante

Seu `useLiveRoom` já cria uma `RTCPeerConnection` quando recebe:

```

</> TypeScript

```



```
student-media-offer
```

e inclusive adiciona o stream do professor nela:

```
if (mediaStream) {  
  mediaStream.getTracks().forEach((track) => {  
    pc.addTrack(track, mediaStream);  
  });  
}
```

</> TypeScript



useLiveRoom

Depois, no StreamLive , ao clicar em **Aprovar**, você pega essa mesma conexão e faz novamente:

```
mediaStream.getTracks().forEach((track) => {
```

</> TypeScript



StreamLive

E depois cria outro offer .

Então a arquitetura atual é aproximadamente:

```
Aluno pede câmera  
↓  
useLiveRoom cria PC  
↓  
PC recebe offer do aluno  
↓  
Professor responde automaticamente  
↓  
Professor clica "Aprovar"  
↓  
StreamLive usa o MESMO PC  
↓  
adiciona tracks novamente  
↓  
cria OUTRO offer  
↓  
Aluno responde  
↓  
Professor não trata answer
```



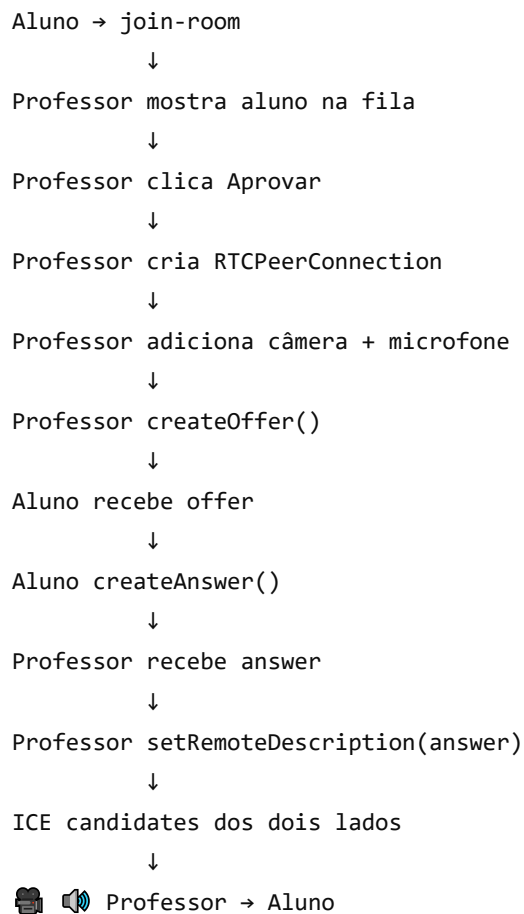
Isso é bastante propenso a quebrar a negociação.

Como eu corrigiria

Eu recomendo deixar uma única parte do código responsável pela sinalização WebRTC.

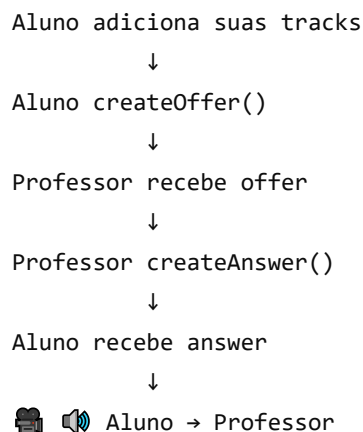
A estrutura deveria ser:

Professor



```
graph TD; A[Aluno → join-room] --> B[Professor mostra aluno na fila]; B --> C[Professor clica Aprovar]; C --> D[Professor cria RTCPeerConnection]; D --> E[Professor adiciona câmera + microfone]; E --> F[Professor createOffer()]; F --> G[Aluno recebe offer]; G --> H[Aluno createAnswer()]; H --> I[Professor recebe answer]; I --> J[Professor setRemoteDescription(answer)]; J --> K[ICE candidates dos dois lados]; K --> L[📹 🗣️ Professor → Aluno];
```

E, separadamente, quando o aluno solicitar câmera/microfone:



```
graph TD; A[Aluno adiciona suas tracks] --> B[Aluno createOffer()]; B --> C[Professor recebe offer]; C --> D[Professor createAnswer()]; D --> E[Aluno recebe answer]; E --> F[📹 🗣️ Aluno → Professor];
```

Correção mínima que falta

No `useLiveRoom.ts`, você precisa adicionar um handler de `answer`.

Algo nessa linha:

```
.on('broadcast', { event: 'answer' }, async ({ payload }: any) => {
  if (!isMounted || !isTeacher) return;

  const { answer, studentId } = payload;

  const pc = peerConnectionsRef.current[studentId];

  if (!pc || !answer) return;

  try {
    console.log('📡 [PROFESSOR] Answer recebida do aluno:', studentId);

    if (pc.signalingState === 'have-local-offer') {
      await pc.setRemoteDescription(
        new RTCSessionDescription(answer)
      );

      console.log('✅ [PROFESSOR] Answer aplicada:', studentId);
    }
  } catch (error) {
    console.error(
      '❌ [PROFESSOR] Erro ao aplicar answer:',
      error
    );
  }
});
```

</> TypeScript



Isso precisa ficar junto dos outros `.on('broadcast', ...)`.

Mas eu faria mais uma alteração

No `useLiveRoom`, você atualmente cria a conexão ao receber `student-media-offer` e já coloca o vídeo do professor nela:

```
if (!pc) {
  pc = new RTCPeerConnection(servers);
  peerConnectionsRef.current[studentId] = pc;
```

```
if (mediaStream) {  
  mediaStream.getTracks().forEach((track) => {  
    pc.addTrack(track, mediaStream);  
  });  
}
```

</> TypeScript



Isso significa que o professor pode começar a negociar antes mesmo de clicar em **Aprovar**.  useLiveRoom

Eu mudaria essa lógica para que:

student-media-offer

Apenas coloque o aluno na fila:

```
.on('broadcast', { event: 'student-media-offer' }, ({ payload }) => {  
  if (!isMounted || !isTeacher) return;  
  
  const { studentId } = payload;  
  
  setPendingStudents(prev =>  
    prev.includes(studentId)  
      ? prev  
      : [...prev, studentId]  
  );  
})
```

</> TypeScript



Não cria PC aqui.

A conexão WebRTC do professor deve ser criada somente dentro do:

</> TypeScript



Assim você tem uma negociação limpa.

Outro detalhe

No aluno, você está enviando:

</> TypeScript



e depois espera:

</> TypeScript



Isso é uma negociação separada da negociação professor → aluno.

Está correto conceitualmente, mas você precisa garantir que cada `RTCPeerConnection` tenha **um único papel por negociação** e que os `offer/answer` sejam processados pelo mesmo PC.

O `ontrack` do aluno está correto

Esta parte:

```
pc.ontrack = (event) => {  
  if (event.streams && event.streams[0]) {  
    if (remoteVideoRef.current) {  
      remoteVideoRef.current.srcObject = event.streams[0];  
      remoteVideoRef.current.play();  
    }  
  }  
}
```

</> TypeScript



está no lugar certo.  StudentLive.tsx

Então o problema **não é simplesmente o `<video>` do aluno.**

Quando o professor finalmente enviar as tracks e a negociação terminar, esse `ontrack` deverá receber o `MediaStream`.

E tem uma coisa boa

O professor já está fazendo:

```
mediaStream.getTracks().forEach((track) => {
```

</> TypeScript



portanto ele está tentando enviar **vídeo e áudio** pela mesma conexão.  StreamLive

E o `getUserMedia` também solicita os dois:

```
const stream = await navigator.mediaDevices.getUserMedia({  
  video: { width: 1280, height: 720 },
```

 TypeScript



 StreamLive

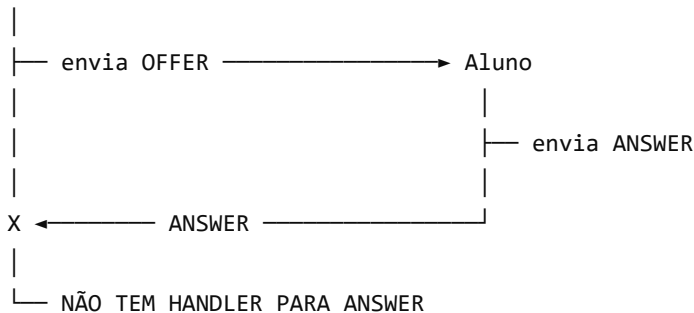
Ou seja, **não falta capturar áudio/vídeo do professor.**

O problema está na **negociação/signaling da PeerConnection**.

Resumo

O bug central é:

Professor



E há um segundo problema arquitetural: `useLiveRoom` e `StreamLive` estão **mexendo na mesma PeerConnection e fazendo negociações diferentes**.

Eu **corrigiria os dois juntos**, em vez de apenas adicionar o listener de `answer`, porque senão provavelmente você vai continuar encontrando erros como `InvalidStateError`, `wrong state`, `have-local-offer / have-remote-offer` ou renegociações quebradas.

Se quiser, eu posso te devolver  **os `useLiveRoom.ts`, `StudentLive.tsx` e `StreamLive.tsx` já corrigidos completos**, mantendo sua interface atual e fazendo professor → aluno (vídeo + áudio) e aluno → professor funcionarem.

Fontes



então, corrija e adicione essa parte de autorização pro aluno entrar na sala também além dos pedidos de comunicação, me entregue os arquivos na integra

Sim. Corrigi a arquitetura para separar **duas autorizações diferentes**:

1. **Entrada na sala:** o aluno envia `join-room` → professor vê o pedido → professor **autoriza ou recusa**.
2. **Comunicação de câmera/microfone:** depois de entrar, o aluno pode solicitar câmera/microfone → professor autoriza → começa a negociação WebRTC.
3. Corrigi também o problema principal do vídeo/áudio: o professor agora recebe o `answer` do aluno e aplica corretamente na `RTCPeerConnection`. No código original, o aluno enviava o `answer`, mas não havia listener correspondente no professor.

 `StudentLive.tsx`

 `useLiveRoom`

4. Removi a negociação duplicada que estava acontecendo entre `useLiveRoom` e `StreamLive`.
5. Adicionei armazenamento temporário de ICE candidates para evitar perder candidatos que chegam antes da `remoteDescription`.
6. Mantive o fluxo de mídia do professor para o aluno e do aluno para o professor. O professor já captura áudio e vídeo juntos no `getUserMedia`, então essa parte não precisava ser substituída.  `StreamLive`

Arquivos completos

1. `StudentLive.tsx`

 **Baixar `StudentLive.tsx`**

2. `StreamLive.tsx`

 **Baixar `StreamLive.tsx`**

3. `useLiveRoom.ts`

 **Baixar `useLiveRoom.ts`**

O `StreamHeader.tsx` não precisa ser alterado.

Fluxo agora



Isso também evita o problema do código anterior em que `student-media-offer` já fazia o `useLiveRoom` criar uma conexão e responder automaticamente, enquanto o `StreamLive` tentava negociar a mesma conexão novamente. `useLiveRoom` `StreamLive`

Importante: os arquivos estão completos, não são apenas trechos. `</>`

`</>` **StudentLive.tsx**
Código

`</>` **StreamLive.tsx**
Código

`</>` **useLiveRoom.ts**
Código

Fontes



O ChatGPT pode cometer erros. Por isso, lembre-se de conferir informações relevantes.